

We thought we could bring forth something unique by combining a really great AI processor and a GPU together in a way no other AI startup was doing.

But for certain reasons, I also took over the business side—operations, HR, and legal stuff. And I enjoyed that kind of work as well. In a small company, you get to do these things from scratch. You get exposed to the details of everything. It's very refreshing. It's a big contrast from a big company.

How are AI programs different from traditional ones?

So, first of all, AI programs are very different from regular programs. In those programs, there's a serial or sequential flow. You have some branches back and forth. You may have many processors, but each one is running threads. It's easy for humans to read it because humans write the code.

AI programs say something like this, "Take some information, represent it like an image or a very long string of sentences, and then multiply it by a very large array of numbers, and then do that a thousand times." As you multiply by the numbers, you're finding out the associations of bad information with previously stored information in some subtle but distributed way. It goes through two steps; you train the model (the set of operations is called a model) and you have an expected result.

Say, I want to complete this sentence, or I want to identify an object in a picture. When you start the model, it has no information in it. So, as you train the model, it starts to understand the relationship between the inputs and the stored information. And that's the AI revolution.

Why do you feel we need to go above and beyond GPUs when it comes to AI processing?

The number of calculations you do in AI programs is very large. As

it turns out, GPUs were better at running lots of math than regular CPUs. GPUs are actually built to run programs on pixels, which are independent. It was not a bad start and, obviously, people had real success with speeding that up.

If you actually look at the code for GPT3—when they trained it, they used five to ten thousand GPUs in a very large cluster. That must have cost something like a hundred million dollars! Also, the program itself is probably just a thousand lines of PyTorch. So, there are more GPUs than lines of code!

TODAY THE AI REVOLUTION IS BIG AND WE'RE GOING TO BRING SOMETHING INTERESTING WITH THE COMBINATION OF GPUS AND AI PROCESSORS

And some of the lines of code, say something like, "Do a matrix multiply that's 10,000 by 10,000"—that's a very large amount of computation. To actually run that program on 10,000 GPUs is very complicated because the GPUs don't just collaborate like 10,000 computers in one big thing. There are multiple layers. There might be about seven to ten layers of software depending on how you define it.

Hence, something different is needed here. For example, one of the things we at Tenstorrent like to do is—you write a thousand lines of code, we have a popular compiler that figures out how to break that problem up on a large number of processors. Our compiler can target from one to many chips. Right now, we're working on the first 256 chips

and we're going to work our way up to 1000, which we think would be an interesting number for these kinds of training problems.

What, according to you, is the right way to balance the power and performance of an AI chip?

Some AI models have very large sections of data. You would think making a really big RAM and putting the processing next to it would work. The problem with that is that every time you want to read the data, it has to read across the big RAM, which is a high-power process.

So, the other way to do it is to take the data and break it into small pieces, and then put the processing next to the small piece. That's how you get the power efficiency of having the data local to the processing, and not having to go so far across the chip—because a lot of power is used in moving data across the chip.

And you want the data and the processing to be local, but you also want enough data there to be interesting from a computing point of view. So, that's one part. The other is that you want the data from one computation to go right to the next computation. You want to keep all the data on the chip and have it move through the pipeline without getting stuck, delayed, or written to memory. So, these two steps make the computation much more power-efficient.

Most AI systems suffer from bottleneck problems. How can one create a perfect sync between data sharing and processing?

Keeping most of the data on-chip would solve this issue. The bottleneck is in the processing, and not the memory. So, at the chip level, we can work around that bottleneck by keeping the data on-chip.

At the higher level, in the long run, this is going to be solved by reading data into AI models and